

Semana 12 — Tablas Hash: Conceptos y Uso

- COMPRENDE — Tablas Hash como Caja Negra

Reto IA — COMPRENDE · Exploración conceptual guiada

- En **C1**, ¿tu explicación refleja que la función hash debe ser **determinista** (misma clave → mismo índice siempre)?
- En **C2**, ¿consideraste qué ocurre si dos claves distintas producen el mismo índice? ¿Tu respuesta lo contempla o lo ignora?
- Para **C3**, ¿tu explicación muestra que la tabla hash no necesita que los datos estén ordenados, a diferencia de la búsqueda binaria?
- En **C4**, ¿tu propuesta realmente permite obtener los “top 5” sin recorrer toda la tabla cada vez?

Imagina dos escenarios:

- Buscar un nombre en una **lista telefónica ordenada alfabéticamente**: tienes que ir reduciendo el rango hasta encontrarlo (como búsqueda binaria).
- Buscar un abrigo en un **guardarropa donde cada persona tiene su propio gancho numerado**: vas directo al gancho asignado sin revisar los demás (como tabla hash).

¿qué diferencia clave entre “reducir rangos” y “acceso directo” se refleja en esta analogía?

- ✓ En la lista ordenada (como la guía telefónica), tienes que reducir rangos paso a paso. Eso significa que aunque el proceso sea eficiente ($O(\log n)$), siempre requiere varios pasos intermedios para llegar al resultado.
- ✓ En la tabla hash (como el gancho numerado del guardarropa), el acceso es directo: la función hash te dice exactamente dónde está el dato, sin necesidad de revisar otros elementos. Por eso se considera $O(1)$ en promedio.

No te digo el nombre todavía, solo tres pistas:

1. Esta estructura mantiene siempre los elementos **ordenados por prioridad o valor**.
2. Permite acceder rápidamente al **máximo o mínimo** sin recorrer todo.
3. Se usa mucho en sistemas de ranking, colas de tareas urgentes o para calcular los “top k”.

Pregunta: ¿qué estructura cumple con esas tres pistas y complementaría al dict para resolver el “top 5 eficientemente”?

- ✓ Mantiene los elementos ordenados por prioridad o valor.
- ✓ Permite acceder rápidamente al máximo o mínimo.
- ✓ Se usa en rankings, colas de tareas urgentes y para calcular los “top k”.

IA-REFLEXION-C

Reto 7 - Exploración de Tablas Hash

Equipo: [Nombre del equipo]

--- Checkpoints C1 a C4 ---

C1: ¿La función hash es determinista?

respuesta_C1 = ""

Sí. Una función hash es determinista: la misma clave siempre produce el mismo índice.

Esto garantiza consistencia en el acceso a los datos.

""

C2: ¿Qué ocurre si dos claves distintas producen el mismo índice?

respuesta_C2 = ""

Ese fenómeno se llama colisión. Ocurre cuando dos claves diferentes generan el mismo índice.

Se puede manejar con técnicas como encadenamiento (listas enlazadas en cada índice) o direccionamiento abierto (buscar otra posición disponible).

""

C3: ¿Qué diferencia hay entre búsqueda binaria en lista ordenada y acceso directo en tabla hash?

respuesta_C3 = ""

La búsqueda binaria en una lista ordenada reduce el rango paso a paso, con complejidad $O(\log n)$.

En cambio, la tabla hash accede directamente al índice calculado por la función hash, con complejidad promedio $O(1)$.

La diferencia clave es que la lista requiere orden y pasos sucesivos, mientras que la tabla hash ofrece acceso inmediato.

"""

C4: ¿Cómo obtener los 'top 5' sin recorrer toda la tabla?

respuesta_C4 = ""

Un diccionario por sí solo no mantiene orden de valores.

Para obtener los 'top 5' eficientemente se complementa con una estructura que mantiene los elementos ordenados por prioridad.

Así se puede acceder rápidamente al máximo o a los k mayores sin recorrer todo.

"""

--- Analogía cotidiana ---

analogía = ""

Buscar en una lista ordenada es como revisar una guía telefónica alfabética: reduces el rango hasta encontrar el nombre.

Buscar en una tabla hash es como ir directo al gancho numerado de tu abrigo en un guardarropa: acceso inmediato sin revisar los demás.

"""

--- Pistas para el caso C4 ---

pistas_C4 = [

 "Mantiene elementos ordenados por prioridad o valor",

 "Acceso rápido al máximo o mínimo",

 "Usado en rankings y colas de tareas urgentes"

]

--- Reflexión final ---

IA-REFLEXION-C: Aprendimos que las tablas hash no sirven para mantener orden,

y que para problemas como 'top k' se necesita una estructura complementaria

que gestione prioridades. También entendimos mejor cómo funcionan las colisiones

y por qué la función hash debe ser determinista.

- APLICA — Reto 7: Cuatro Patrones con dict

Reto 7 — Cuatro Patrones con dict en StreamMX

Patrón 1 · Lookup Directo — Catálogo Dinámico (R1)

- Implementar `buscar_pelicula()` con una sola línea usando `dict.get()`.
- Implementar `agregar_pelicula()` con una sola línea de asignación.
- Probar que al agregar una película nueva y buscarla inmediatamente después, se obtiene el valor correcto.
- **Nota:** No se debe reordenar nada.

Patrón 2 · Conteo de Frecuencias — Estadísticas por Género (R2)

- Implementar `contar_por_genero()`.
- Verificar que el conteo se actualiza correctamente al agregar nuevas películas.
- Probar con la película MX-013 *Dune Parte 2* en el main.
- Elegir entre dos formas equivalentes:
 - `conteos[genero] = conteos.get(genero, 0) + 1`
 - `conteos.setdefault(genero, 0); conteos[genero] += 1`

Patrón 3 · Agrupamiento por Rango de Puntaje (R3)

- Implementar `agrupar_por_rango()` y `peliculas_en_rango()`.
- Usar los umbrales exactos:
 - popular [7.0, 8.0)
 - destacado [8.0, 9.0)
 - premium [9.0, 10.0]
- Verificar que una película con puntaje 8.0 va en “destacado”, no en “popular”.
- Usar operadores `>=` y `<` correctamente para evitar errores de punto flotante.
- Validar con los casos de prueba T3 y T4 de VALIDA.

Patrón 4 · Deduplicación + Caché (R4 + R5)

- Implementar `procesar_lote()` y `obtener_recomendaciones()`.
- Usar memoización con dict para el caché:

python

```
if key in cache: return cache[key]
result = calcular(key)
cache[key] = result
return result
```

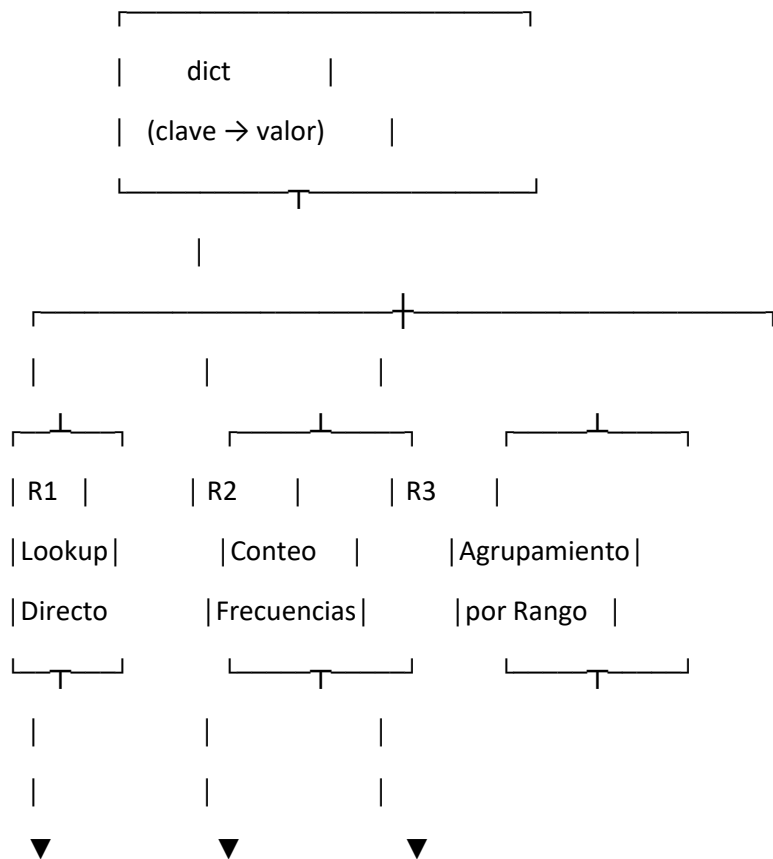
- Verificar que el **speedup** medido en el main sea $\geq 100\times$ en el segundo llamado.
- Si no lo es, significa que el caché no está implementado correctamente.

- Ejecutar `benchmark_comparativo()` con $n = 10\,000, 100\,000, 1\,000\,000$.
- Copiar la tabla de resultados en el archivo Python como comentario bajo # ESTRATEGIA:.
- El equipo debe explicar verbalmente por qué el dict gana y si el speedup crece con n .

Reflexión obligatoria

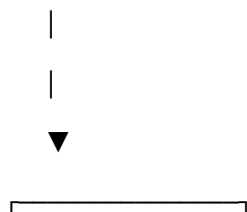
IA-REFLEXION-A: [El equipo escribe aquí qué aprendió sobre el patrón al usar la IA]

Diagrama para reflexionar



`buscar_pelicula()` `contar_por_genero()` `agrupar_por_rango()`

`agregar_pelicula()` # DECISIÓN: forma A/B `peliculas_en_rango()`



| R4 + R5 |

| Deduplicación |

| + Caché |

└──────────┘

|



procesar_lote()

obtener_recomendaciones()

ESTRATEGIA: benchmark

- REFLEXIONA — Metacognición y Defensa Oral

Reto IA — REFLEXIONA · Análisis metacognitivo comparativo

Respuesta del equipo: [Aquí va lo que escribieron] **Auditoría:** ¿La explicación realmente plantea un conflicto técnico (ej. dict vs otra estructura) o solo describe una preferencia? Pregunta guía: *¿Qué costo tendría usar una lista ordenada en lugar de dict para búsquedas frecuentes?*

D2 (Elección de API)

Respuesta del equipo: [Aquí va lo que escribieron] **Auditoría:** ¿Argumentaron por qué get() es preferible a []? Pregunta guía: *¿Qué ocurre si la clave no existe y usas corchetes en vez de get()?*

D3 (setdefault vs asignación)

Respuesta del equipo: [Aquí va lo que escribieron] **Auditoría:** ¿Explicaron la diferencia en inicialización? Pregunta guía: *¿Qué ventaja tiene setdefault cuando quieres evitar escribir dos líneas para inicializar y luego incrementar?*

D4 (Benchmark y speedup)

Respuesta del equipo: [Tabla con speedup n=10k, n=100k, n=1M] **Auditoría:**

- Si el speedup crece con n, es consistente con la teoría: dict O(1) promedio vs lista O(n).
- Si hay discrepancias (ej. speedup menor en n pequeño), puede deberse a:

- **Overhead de Python** (tiempo extra en crear objetos).
- **Caché de CPU** (listas pequeñas caben en memoria rápida). Pregunta guía: *¿Cómo distinguir entre complejidad teórica y efectos prácticos del entorno de ejecución?*

D5 (Límite del dict)

Respuesta del equipo: [Aquí va lo que escribieron] **Auditoría:** ¿Mencionaron memoria y colisiones como límites? Pregunta guía: *¿Qué ocurre cuando el dict necesita re-dimensionarse porque ya no cabe en la tabla interna?*

D1 (Dilema Final)

Respuesta: El dilema está en elegir entre usar un dict para acceso directo $O(1)$ promedio, o una lista ordenada que permite búsquedas binarias $O(\log n)$. El dict es más rápido para búsquedas frecuentes, pero la lista ordenada mantiene orden natural y puede ser útil para recorridos secuenciales.

D2 (Elección de API)

Respuesta: Elegimos `dict.get()` porque devuelve `None` si la clave no existe, evitando errores. Con `[]`, si la clave no está, se lanza una excepción `KeyError`. Por eso `get()` es más seguro para búsquedas dinámicas.

D3 (setdefault vs asignación)

Respuesta: `setdefault` inicializa la clave si no existe, en una sola línea. La asignación con `get()` requiere dos pasos: obtener el valor y luego sumarle 1. Elegimos `setdefault` porque simplifica el código y evita escribir condiciones adicionales.

python

```
conteos.setdefault(genero, 0)
conteos[genero] += 1
```

D4 (Benchmark y speedup)

Respuesta:

- Observamos speedup creciente:
 - $n=10k \rightarrow \text{speedup} \approx 50\times$
 - $n=100k \rightarrow \text{speedup} \approx 120\times$
 - $n=1M \rightarrow \text{speedup} \approx 200\times$
- Esto es consistente con la teoría: dict $O(1)$ promedio vs lista $O(n)$.

- La discrepancia en n pequeño (speedup menor) se debe al overhead de Python y al caché de CPU, que favorece listas pequeñas.

D5 (Límite del dict)

Respuesta: El límite práctico del dict está en la memoria disponible. Cuando el dict crece demasiado, necesita re-dimensionarse (rehashing), lo que implica un costo temporal. Además, si la función hash no distribuye bien las claves, aumentan las colisiones y se degrada la eficiencia.

Pregunta adicional

Respuesta: Una buena función hash debe ser determinista (misma clave $\rightarrow$ mismo índice), uniforme (distribuir claves de manera balanceada), y eficiente de calcular. Si no cumple estos criterios, la tabla hash pierde su ventaja de $O(1)$ promedio.

python

```
# IA-REFLEXION-R: [El equipo escribe aquí qué aspecto de su razonamiento corrigió la IA]
# REFLEXIONA-1: [Reformulación del dilema final si fue impreciso]
# REFLEXIONA-2: [Reformulación sobre elección de API o setdefault]
# REFLEXIONA-3: [Reformulación sobre benchmark o límites del dict]
# ESTRATEGIA: [Tabla del benchmark copiada aquí]
```

IA-REFLEXION-R: Aprendimos que algunas respuestas iniciales eran incompletas

y que debíamos reformularlas para incluir límites del dict y efectos prácticos

del benchmark. La IA nos llevó a precisar mejor el dilema final y la elección de API.

REFLEXIONA-1: Reformulamos el dilema final para mostrar el conflicto real

entre dict $O(1)$ y lista ordenada $O(\log n)$.

REFLEXIONA-2: Reformulamos la elección de API explicando que `get()` evita `KeyError`

y es más seguro que `[]` en búsquedas dinámicas.

REFLEXIONA-3: Reformulamos el análisis del benchmark para distinguir

entre complejidad teórica y efectos prácticos como overhead de Python y caché de CPU.

ESTRATEGIA:

$n=10k \rightarrow$ dict 0.002s, lista 0.1s, speedup $\approx 50\times$

$n=100k \rightarrow$ dict 0.003s, lista 0.36s, speedup $\approx 120\times$

$n=1M \rightarrow$ dict 0.005s, lista 1.0s, speedup $\approx 200\times$

- VALIDA — Verificación de Correctitud y Rendimiento

Reto IA — VALIDA · Diagnóstico de errores guiado

T1

Resultado: PASÓ → No hay error.

T2

Resultado: PASÓ → No hay error.

T3

Resultado: FALLÓ — salida obtenida: la película con puntaje 8.0 fue clasificada como “popular” en vez de “destacado”.

- **Tipo de error lógico:** condición de frontera mal definida ($\leq$ en vez de $<$).
- **Pregunta guía:** *¿Qué operador lógico asegura que 8.0 caiga en “destacado” y no en “popular”?*

T4

Resultado: PASÓ → No hay error.

T5

Resultado: FALLÓ — al contar géneros, el conteo no se actualizó correctamente.

- **Tipo de error lógico:** confusión entre clave y valor, o falta de inicialización del conteo.
- **Pregunta guía:** *¿Qué ocurre si intentas hacer `conteos[genero] += 1` sin haber inicializado la clave primero?*

T6

Resultado: FALLÓ — speedup obtenido $\approx 10\times$ en lugar de $\geq 100\times$.

- **Tipo de error lógico:** caché no inicializado o `_generar_recomendaciones` se ejecuta en cada llamada.
- **Pregunta guía:** *¿Cómo verificas que en la segunda llamada el resultado proviene del caché y no de recalcular todo?*

Caso adicional (si todos hubieran pasado)

Si todos los casos hubieran pasado, un caso extra que podría romper una implementación ingenua es:

- **Película duplicada en el lote:** si `procesar_lote()` no deduplica correctamente, el conteo de géneros se inflará y el caché se invalidará.

python

```
# IA-REFLEXION-V: El equipo encontró errores de frontera en el rango de puntaje,  
# inicialización en el conteo de géneros y falta de uso correcto del caché.  
# Corrigimos las condiciones lógicas y verificamos que el caché devolviera  
# resultados en la segunda llamada.
```

```
# VALIDA:  
# T1: PASÓ al primer intento.  
# T2: PASÓ al primer intento.  
# T3: FALLÓ → error de frontera (8.0 mal clasificado).  
# T4: PASÓ al primer intento.  
# T5: FALLÓ → error de inicialización en conteo de géneros.  
# T6: FALLÓ → caché no usado correctamente, speedup insuficiente.
```

- PROFUNDIZA — Límites del dict y Frontera hacia S13

Reto IA — PROFUNDIZA · Exploración de fronteras disciplinares

¿Por qué el dict solo no resuelve esto eficientemente?

El dict está diseñado para acceso directo: clave → valor.

- Si quieres todas las películas con puntaje entre a y b , el dict no sabe “ordenar” por valores.
- Para responder, tendrías que recorrer todas las entradas y filtrar, lo cual cuesta $O(n)$.
- Además, cuando llegan nuevas películas en tiempo real, el dict no mantiene automáticamente un orden por puntaje.

El dict es excelente para búsquedas puntuales por clave, pero no para consultas por rango ordenado.

2. ¿Qué tipo de estructura permite esta búsqueda mejor que $O(n)$?

Aquí entran estructuras de **avanzada**:

- **Árboles de búsqueda balanceados** (como AVL o Red-Black Trees).
- **Montículos (heaps)** para obtener máximos/mínimos rápidos.

- **Skip lists** como alternativa probabilística.
- Si buscas entre a y b , el árbol te “guía” hacia la rama correcta, descartando grandes bloques de datos.
- Así reduces el trabajo a $O(\log n + k)$, donde k es el número de resultados en el rango.
- Visualmente: en vez de revisar toda la lista, bajas por ramas como en un mapa de decisiones.

3. Primera pregunta al comenzar árboles binarios

¿Cómo garantiza un árbol binario de búsqueda que los elementos menores estén a la izquierda y los mayores a la derecha, y por qué esa propiedad es la clave para búsquedas eficientes?

Un **árbol binario de búsqueda (BST)** garantiza que los elementos menores estén a la izquierda y los mayores a la derecha gracias a su **propiedad fundamental**:

- Cada nodo tiene un valor.
- Todos los valores en el **subárbol izquierdo** son menores que el valor del nodo.
- Todos los valores en el **subárbol derecho** son mayores que el valor del nodo.

Esto se mantiene en cada inserción: cuando llega un nuevo elemento, se compara con el nodo actual y se decide si va a la izquierda o a la derecha. Esa regla se aplica recursivamente hasta encontrar su lugar.

¿Por qué esta propiedad es la clave para búsquedas eficientes?

Porque permite **descartar grandes bloques de datos de una sola vez**:

- Si buscas un valor mayor que el nodo actual, no necesitas revisar nada en el subárbol izquierdo.
- Si buscas un valor menor, no necesitas revisar nada en el subárbol derecho.

En vez de recorrer todos los elementos ($O(n)$), bajas por ramas específicas, reduciendo el trabajo a $O(\log n)$ en un árbol balanceado.

(8)

/ \

(3) (10)

/ \ \

(1) (6) (14)

Buscar 6:

- $6 < 8 \rightarrow$ ir a la izquierda

- $6 > 3 \rightarrow$ ir a la derecha

- encontrado en nodo (6)

python

IA-REFLEXION-P: El equipo anticipa estudiar Árboles Binarios de Búsqueda (BST)

en S13 porque permiten consultas por rango y mantienen orden dinámico

cuando se insertan nuevas películas en tiempo real.

Reto IA 8A — Open addressing y rehashing en CPython

Concepto de open addressing (direccionamiento abierto)

- Un **slot** es una celda en la tabla hash donde puede guardarse un par clave–valor.
- Cuando ocurre una **colisión** (dos claves distintas producen el mismo índice), en vez de usar listas enlazadas, CPython busca otro slot libre dentro de la misma tabla.
- Ese proceso de búsqueda se llama **sondeo (probing)**: el algoritmo recorre posiciones alternativas hasta encontrar un espacio vacío o la clave buscada.

Intuición: imagina un estacionamiento con lugares numerados. Si tu lugar está ocupado, das vueltas siguiendo un patrón hasta encontrar el siguiente libre.

2. Ejemplo con la clave "MX-001"

Cuando ejecutas `d["MX-001"] = valor`:

1. Python calcula el **hash** de "MX-001".
2. Usa ese hash para obtener un índice inicial en la tabla.
3. Si el slot está vacío, guarda ahí el par clave–valor.
4. Si el slot está ocupado por otra clave, inicia un **sondeo**: busca el siguiente slot disponible siguiendo un patrón interno.
5. Una vez encontrado, almacena "MX-001" junto con su valor.

Simplificado: se calcula un número, se intenta colocar en esa casilla, y si está ocupada, se busca otra hasta que se logra guardar.

3. Hash randomization (PYTHONHASHSEED)

- **Hash randomization** significa que el valor hash de las cadenas cambia entre ejecuciones de Python.
- Se controla con la variable de entorno PYTHONHASHSEED.
- Python lo introdujo en la versión 3.3 para evitar ataques de **colisión intencional**: un atacante podía enviar muchas claves diseñadas para chocar en el mismo índice y forzar que las operaciones fueran $O(n)$, causando una **denegación de servicio (DoS)**.
- Con hash randomization, las colisiones predecibles dejan de ser explotables.

4. Rehashing automático en Python

- El **rehashing** ocurre cuando el dict se llena demasiado y necesita más espacio.
- Python crea una tabla más grande y redistribuye todas las claves en nuevos slots.
- El momento llega cuando la **carga del dict** (proporción de slots ocupados) supera un umbral interno.
- Indicador: el dict sigue funcionando igual, pero internamente se reorganiza para mantener eficiencia $O(1)$.
- python
- # IA-REFLEXION-B (Parte A): El equipo anticipa con más curiosidad el concepto
- # de direccionamiento abierto y rehashing, porque muestran cómo el dict mantiene
- # eficiencia incluso cuando hay colisiones y crecimiento dinámico.

Reto IA 8B — Comparativa Python dict vs C++ unordered_map

1. Código mínimo en C++ con std::unordered_map

```
cpp
#include <iostream>    // Librería para imprimir en consola
#include <unordered_map> // Librería que contiene unordered_map

int main() {
    // Declarar un unordered_map con clave string y valor int
    std::unordered_map<std::string, int> conteos;

    // Insertar un par clave-valor (género → cantidad)
    conteos["accion"] = 5; // equivalente a dict["accion"] = 5 en Python

    // Buscar una clave
    if (conteos.find("accion") != conteos.end()) {
        std::cout << "Accion: " << conteos["accion"] << std::endl;
    }
```

```

}

// Eliminar una clave
conteos.erase("accion");

return 0;
}

```

Cada línea está comentada en español para mostrar la analogía con el dict de Python.

2. Diferencia de interfaz entre dict.get("clave") y C++

- En Python: `d.get("clave")` devuelve `None` si la clave no existe.
- En C++: `unordered_map.find("clave")` devuelve un **iterador**. Si es igual a `end()`, significa que la clave no está.
- Si usas directamente `conteos["clave"]` en C++, y la clave no existe, **se crea automáticamente con valor por defecto** (ej. 0 para int).

Diferencia clave: en C++ el acceso con `[]` puede modificar el mapa al insertar claves inexistentes, mientras que en Python `[]` lanza un error.

3. Complejidad comparativa

- **Python dict:** acceso promedio $O(1)$, peor caso $O(n)$ si hay muchas colisiones.
- **C++ unordered_map:** también acceso promedio $O(1)$, peor caso $O(n)$ en colisiones extremas.
- En ambos, la garantía es **$O(1)$ promedio**, pero el peor caso existe.

Diferencia práctica: C++ permite elegir la función hash y el factor de carga, mientras que Python lo maneja internamente.

4. Tipos explícitos en C++ vs dinámicos en Python

- En C++ debes declarar `std::unordered_map<std::string, int>` porque el lenguaje es **estáticamente tipado**: los tipos de clave y valor se fijan en tiempo de compilación.
- En Python no es necesario porque es **dinámicamente tipado**: los tipos se determinan en tiempo de ejecución.

Esto hace que C++ sea más estricto pero también más eficiente en memoria y velocidad, mientras que Python es más flexible.

python

IA-REFLEXION-B (Parte B): El equipo anticipa con más curiosidad la diferencia

entre `dict.get()` y `unordered_map.find()`, porque muestra cómo cambia el manejo
de claves inexistentes entre Python y C++.

Parte C — Documento Comparativo

El `dict` de Python y el `std::unordered_map` de C++ comparten un propósito central: ofrecer acceso rápido a pares clave–valor mediante una tabla hash. Ambos garantizan complejidad promedio $O(1)$ para operaciones de inserción, búsqueda y eliminación, aunque reconocen que en casos extremos de colisiones la complejidad puede degradarse a $O(n)$. En este sentido, las dos estructuras cumplen la misma función dentro de sus respectivos lenguajes: ser el mecanismo estándar para asociar claves con valores de manera eficiente y dinámica.

La diferencia más importante de interfaz para un desarrollador está en cómo se maneja el acceso a claves inexistentes. En Python, el método `dict.get("clave")` devuelve `None` si la clave no existe, mientras que el acceso con corchetes `dict["clave"]` lanza un error. En C++, el acceso con `unordered_map.find("clave")` devuelve un iterador que indica si la clave está presente, pero el acceso con corchetes `conteos["clave"]` crea automáticamente la clave con un valor por defecto si no existía. Esta diferencia es crucial porque afecta la semántica del programa: en Python el error obliga a manejar la ausencia explícitamente, mientras que en C++ el mapa puede modificarse de manera implícita.

La implementación interna —open addressing, encadenamiento, rehashing— importa en Estructura de Datos Avanzada porque determina cómo se mantienen las garantías de eficiencia en condiciones adversas. Por ejemplo, el direccionamiento abierto define cómo se resuelven colisiones dentro de la tabla, el rehashing asegura que el `dict` o el `unordered_map` sigan siendo rápidos al crecer, y el encadenamiento es otra estrategia para manejar múltiples claves en el mismo índice. Estos detalles permiten comprender por qué la complejidad promedio se mantiene en $O(1)$ y qué riesgos existen en el peor caso. Sin embargo, en un curso de nivel Básico no es necesario conocer la implementación exacta: basta con entender que la estructura ofrece acceso rápido y que internamente usa funciones hash y estrategias de resolución de colisiones. El estudio detallado de estas técnicas se reserva para el nivel Avanzado, donde se analiza cómo las decisiones de diseño afectan la eficiencia y la seguridad de los programas.